

大数据分析 & 挖掘

ModelArts 在心脏病风险预测中的建模

实验手册



ModelArts 在心脏病风险预测中的建模

一、基本信息

实验名称：ModelArts 在心脏病风险预测中的建模

课程章节：了解数据抽样知识、了解数据标准化和归一化原理

实验分类：大数据分析挖掘

实验时长：90 分钟

实验难度：中级

实验摘要：通过购买 AI 开发平台 ModelArts，并利用开发环境中的 Notebook 开发工具进行过实验。

实验简介：本实验旨在帮您从操作层面了解 ModelArts 的使用方式，并基于 ModelArts 中的开发工具 Notebook 完成实验。本实验从某大型电商的几亿条订单数据随机抽取 1000 条进行清洗和转换处理，统计出客户的好评率。

实验数据：

本实验数据为 UCI 开源数据集 Adult，该数据集包含了 303 条某区域的心脏病检查患者的体测数据，具体字段如下：

字段名	类型	描述
age	STRING	对象的年龄。
sex	STRING	对象的性别，取值为 female 或 male。
cp	STRING	胸部疼痛类型，痛感由重到轻依次为 typical、atypical、non-anginal 及 asymptomatic。
trestbps	STRING	血压。
chol	STRING	胆固醇。
fbs	STRING	空腹血糖。如果血糖含量大于 120mg/dl，则取值为 true，否则取值为 false。

restecg	STRING	心电图结果是否有 T 波，由轻到重依次为 norm 和 hyp。
thalach	STRING	最大心跳数。
exang	STRING	是否有心绞痛。true 表示有心绞痛，false 表示没有心绞痛。
oldpeak	STRING	运动相对于休息的 ST Depression，即 ST 段压值。
slop	STRING	心电图 ST Segment 的倾斜度，程度取值包括 down、flat 及 up。
ca	STRING	透视检查发现的血管数。
thal	STRING	病发种类，由轻到重依次为 norm、fix 及 rev。
status	STRING	是否患病。buff 表示健康，sick 表示患病。

1. 实验需用到的云服务

云服务名称：AI 开发平台 ModelArts

是否预置：否

配置截图：无需截图

实例数量：1

2. 实验消耗预估

云服务	消耗/时	时长
AI 开发平台 ModelArts	0.8 元/时	2 小时

合计：1.6 元

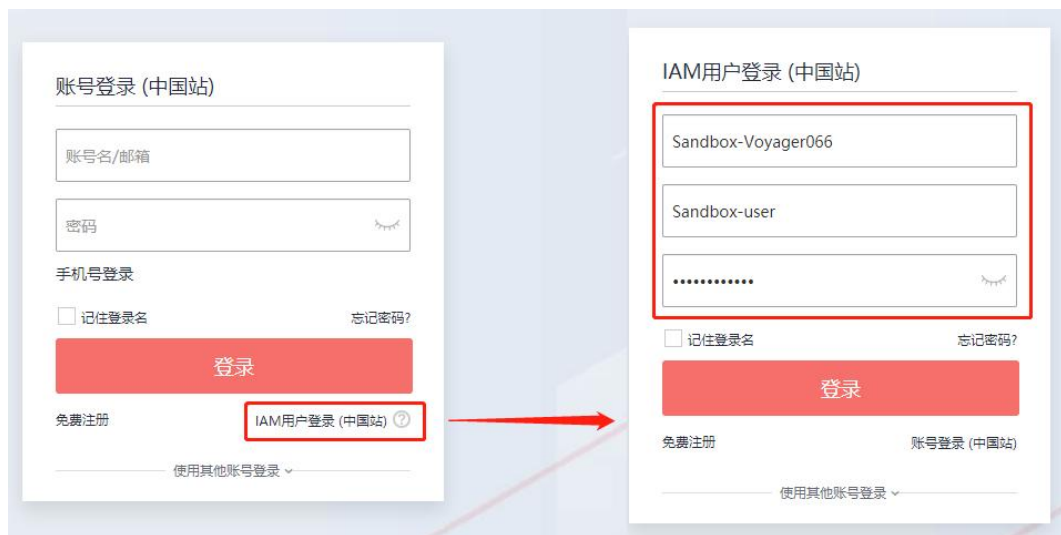
二、前置操作

1. 提前操作

进入【实验操作桌面】，打开浏览器，在地址栏输入 <https://www.huaweicloud.com>，进入华为云页面。点击登录，进入登录页面。



选择【IAM 用户登录】模式，登录对话框中输入系统为您分配的华为云实验账号和密码登录华为云，如下图所示。



三、实验内容

1. 创建 AI 开发平台并上传数据集

1.1. 进入 ModelArts 控制台

具体操作步骤如下：

步骤一：进入首页后点击“产品”，在最左侧产品列表中点击“人工智能”，选择“AI 基础平台”的“AI 开发平台 ModelArts”产品，如图 1-1 所示。

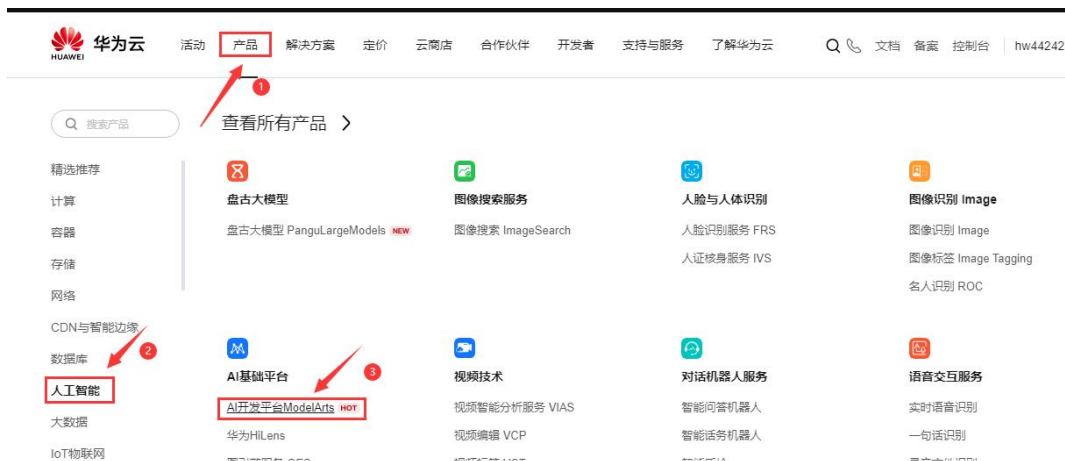


图 1- 1

步骤二：进入 AI 开发平台 ModelArts 后点击“控制台”按钮进入控制台页面，如图 1-2 所示。

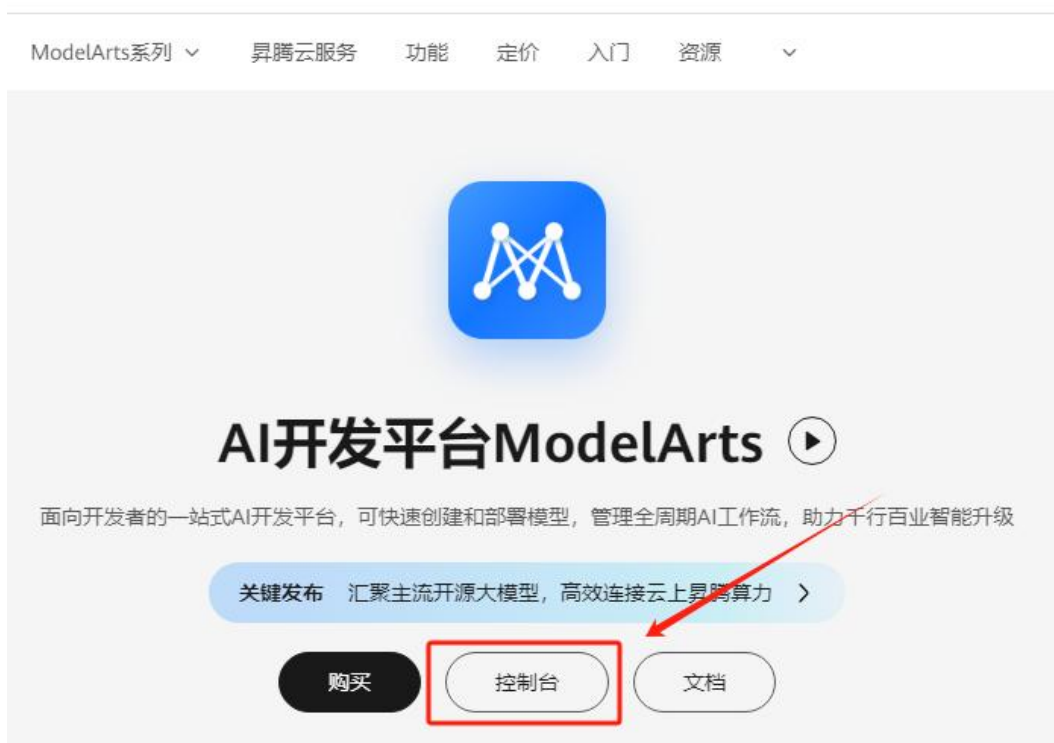


图 1- 2

步骤三：进入控制台界面后，点击“开发环境”选择“Notebook”，如图 1-3 所示。



图 1- 3

1.2. 创建服务

具体操作步骤如下：

步骤一：创建 Notebook，点击“创建”，如图 1-4 所示。



图 1- 4

步骤二：在点击创建后，进入详细配置页面，配置信息如下：

- 名称：自定义
- 时长可以根据需要进行调整
- 其他配置信息默认即可

信息配置完成后，点击“立即创建”按钮，如图 1-5 所示。



图 1- 5

步骤三：接下来会进入“配置信息确认页面”，确认配置信息无误后，点击“提交”按钮完成创建。如图 1-6 所示。

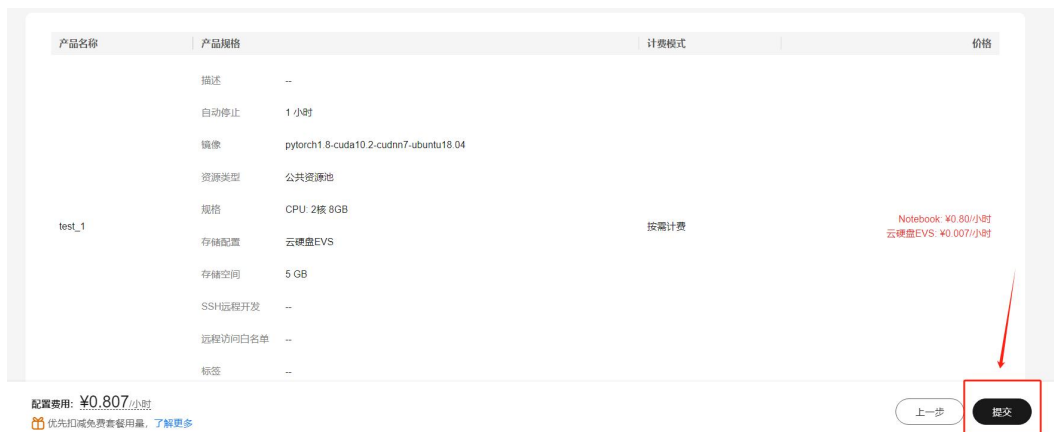


图 1- 6

步骤四：完成创建后显示“任务提交成功！”，点击“立即返回”按钮。如图 1-7 所示。



图 1- 7

返回控制台页面，等待 Notebook 服务创建完成，如图 1-8 所示。



图 1- 8

步骤五：创建完成后，点击服务操作栏的“打开”按钮，如图 1-9 所示。

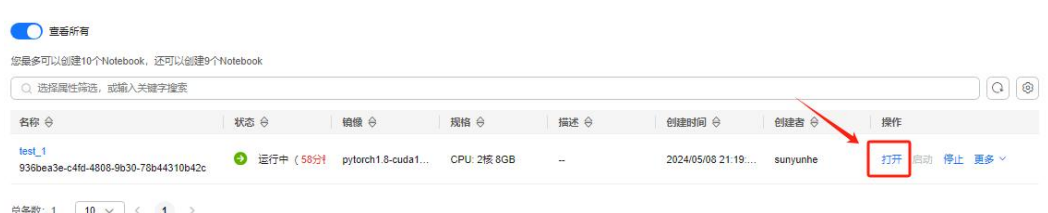


图 1- 9

进入服务后，会出现如图 1-10 所示页面。

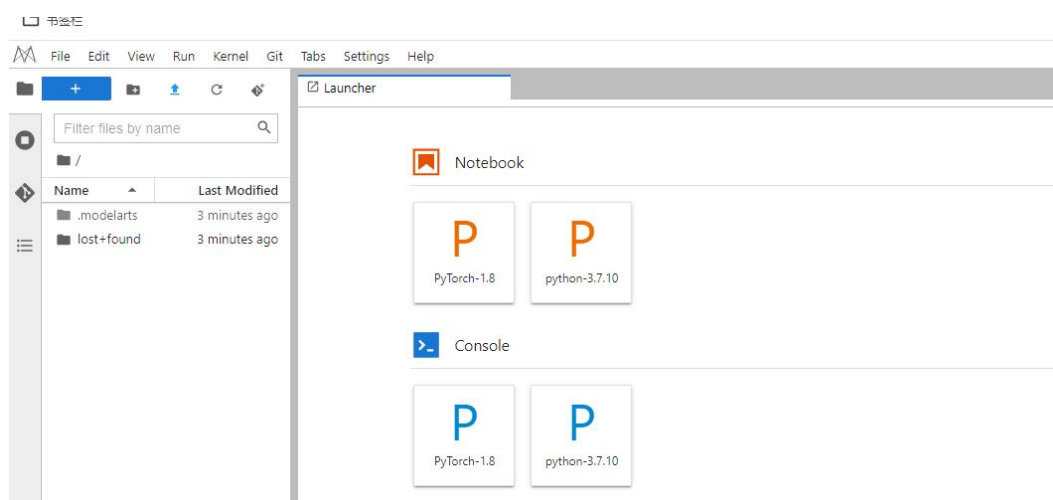


图 1- 10

1.3. 上传数据集

具体操作步骤如下：

步骤一：将本地的数据集上传到 Notebook，如图 1-11 所示。

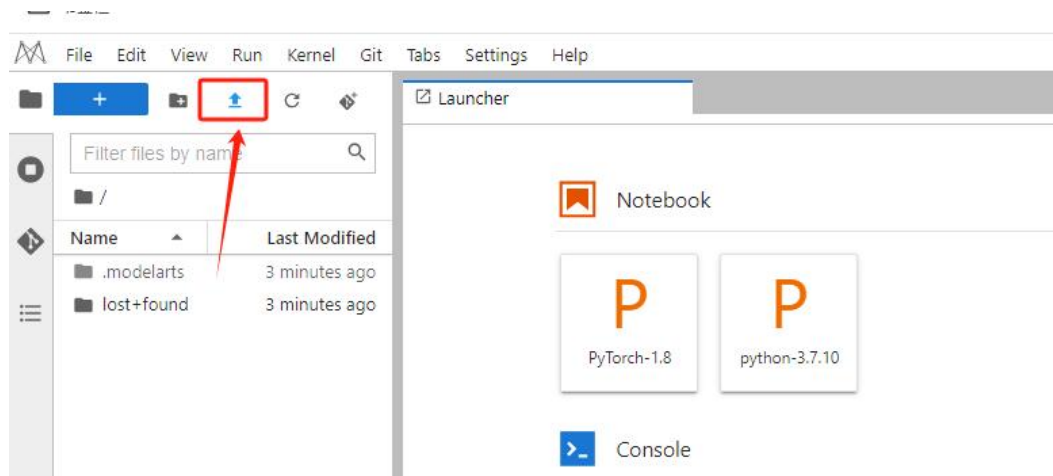


图 1- 11

步骤二：上传文件的方式我们这里选择从本地上传，如图 1-12 所示。



图 1- 12

步骤三：选择本次实验所使用的数据文件，并点击“打开”如图 1-13 所示。

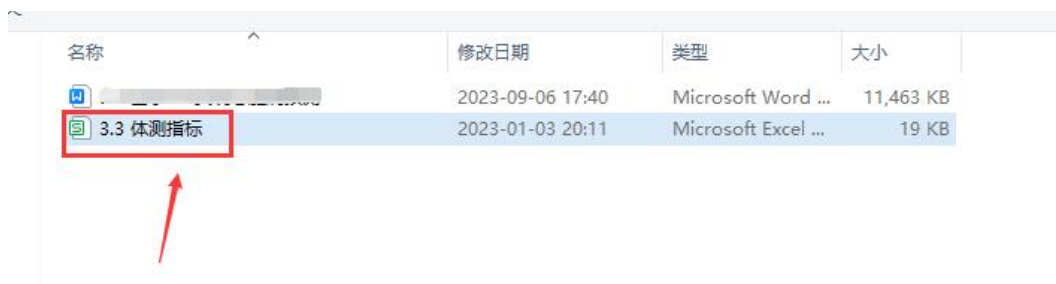


图 1- 13

步骤四：文件上传成功后，等待状态为“文件上传成功”提示，然后关掉上传文件到 Notebook 窗口即可，如图 1-14 所示。



图 1- 14

步骤五：在上传完数据集后，选择内核为“PyTorch-1.8”的开发工具，如下图 1-15 所示。

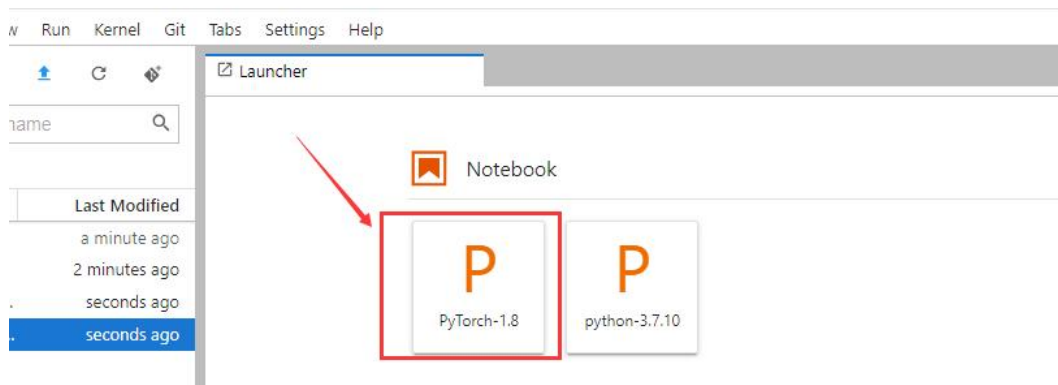


图 1- 15

经过上一步之后出现，如图 1-16 所示的页面。

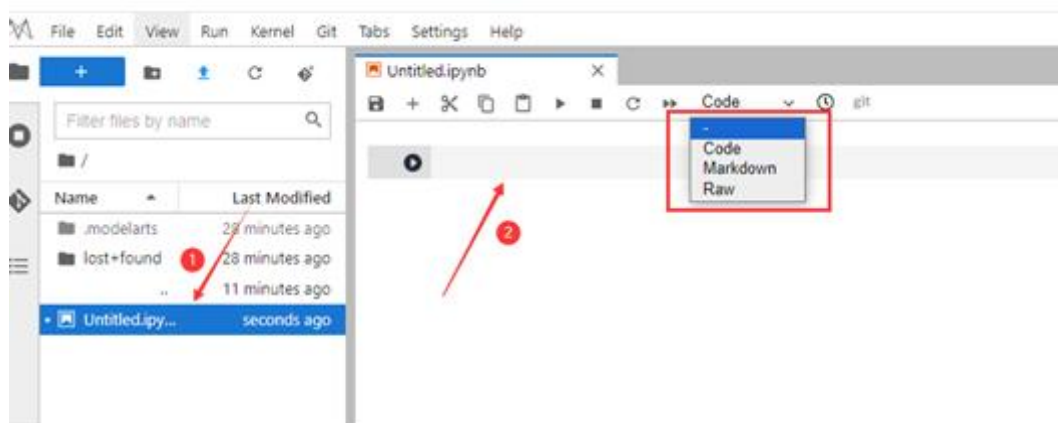


图 1- 16

图 1-16 说明：

- “①”为文件的名字。
- “②”为单元格，此单元格类型有 Code，Markdown，Raw 三种类型。若是在单元格中运行代码，选择 Code 类型。若是运行 Markdown 文本，选择 Markdown 类型。

2. 读取数据并获取数据信息

下面使用 pandas 读取数据表，并查看数据信息。具体操作步骤如下：

步骤一：在单元格中输入以下代码，如图 2-1 所示。

```
import pandas as pd

datas = pd.read_csv("./3.3 体测指标.csv")
datas.info() # 查看摘要信息
# info()获取有关 DataFrame 的信息，包括索引范围，列名、非空值的数量以及每列的数据类型
```



图 2- 1

按 shift+enter 运行单元格，结果如图 2-2 所示。

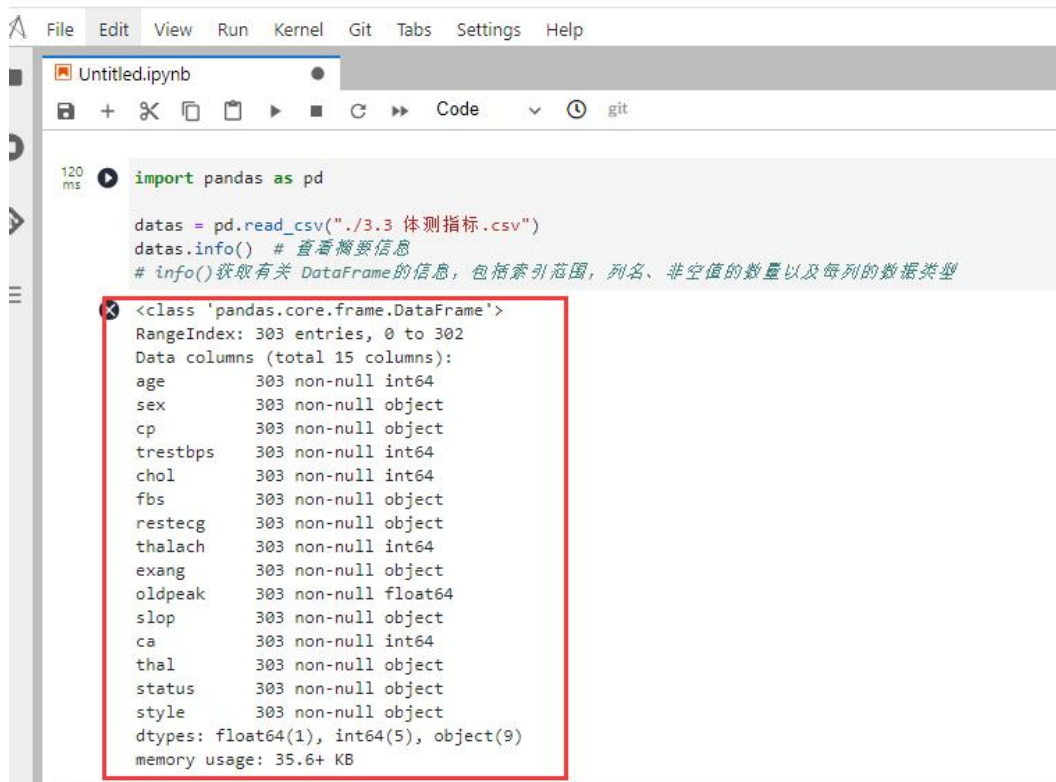


图 2- 2

从运行结果可以看出：本次实验数据共有 303 行，15 列。每一个特征都是非空。float64 数据类型有 1 列，int64 数据类型有 5 列，object 数量类型有 9 列。内存使用情况：35.6+ KB。

步骤二：获得前 5 行数据信息，在单元格中输入以下代码，如图 2-3 所示。

```
datas.head()
```

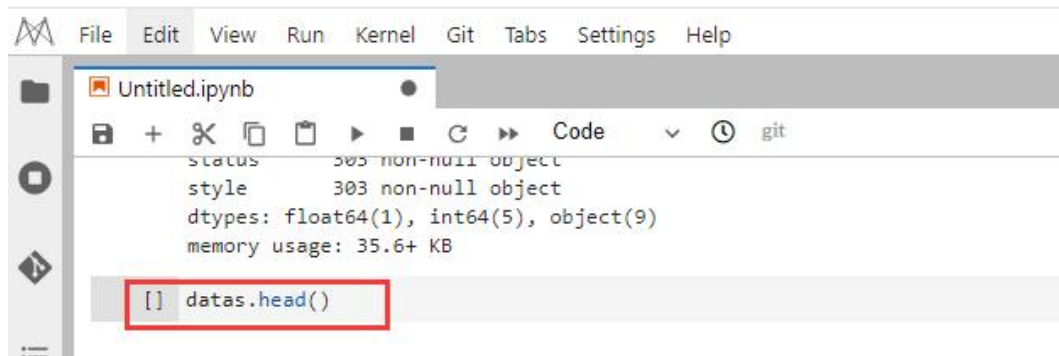


图 2- 3

按 shift+enter 运行单元格，结果如图 2-4 所示。

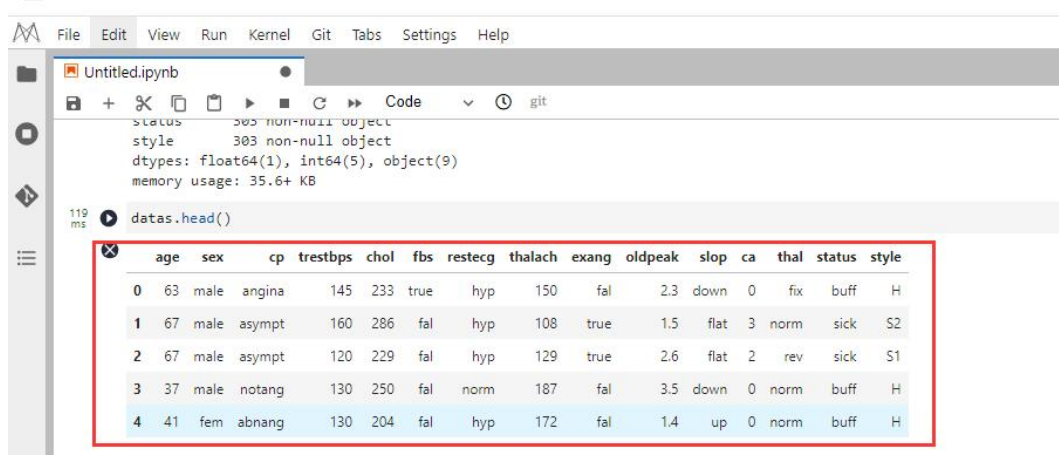


图 2- 4

从运行结果可以看出：前 5 行的数据信息。

步骤三：对"sex", "cp", "fbs", "restecg", "exang", "slop", "thal", "status"字段进行去重，在单元格中输入以下代码，如图 2-5 所示。

```
# 指定读取字段
columns_to_encode = ["sex", "cp", "fbs", "restecg", "exang", "slop", "thal", "status"]
# 使用 for 循环进行遍历，要读取的字段
for column in columns_to_encode:
```

```
print(column) # 输出字段
print(pd.unique(datas["{}".format(column)])) # 对字段去重
```

```
[ ] # 指定读取字段
columns_to_encode = ["sex", "cp", "fbs", "restecg", "exang", "slop", "thal", "status"]
# 使用for循环进行遍历，要读取的字段
for column in columns_to_encode:
    print(column) # 输出字段
    print(pd.unique(datas["{}".format(column)])) # 对字段去重
```

图 2- 5

按 shift+enter 运行单元，结果如图 2-6 所示。

```
sex
['male' 'fem']
cp
['angina' 'asympt' 'notang' 'abnang']
fbs
['true' 'fal']
restecg
['hyp' 'norm' 'abn']
exang
['fal' 'true']
slop
['down' 'flat' 'up']
thal
['fix' 'norm' 'rev']
status
['buff' 'sick']
```

图 2- 6

本次运行的结果显示：

- sex 字段：有'male', 'fem'两种数据。
- cp 字段：有'angina', 'asympt', 'notang', 'abnang'四种数据。其他同理。

3. 特征工程

下面对指定字段进行标签编码和归一化操作。

3.1. 标签编码

操作步骤如下：

步骤一：对"sex", "cp", "fbs", "restecg", "exang", "slop", "thal", "status"字段进行标签编码，在单元格中输入以下代码，如图 3-1 所示。

```
import numpy as np
import sklearn.preprocessing as sp

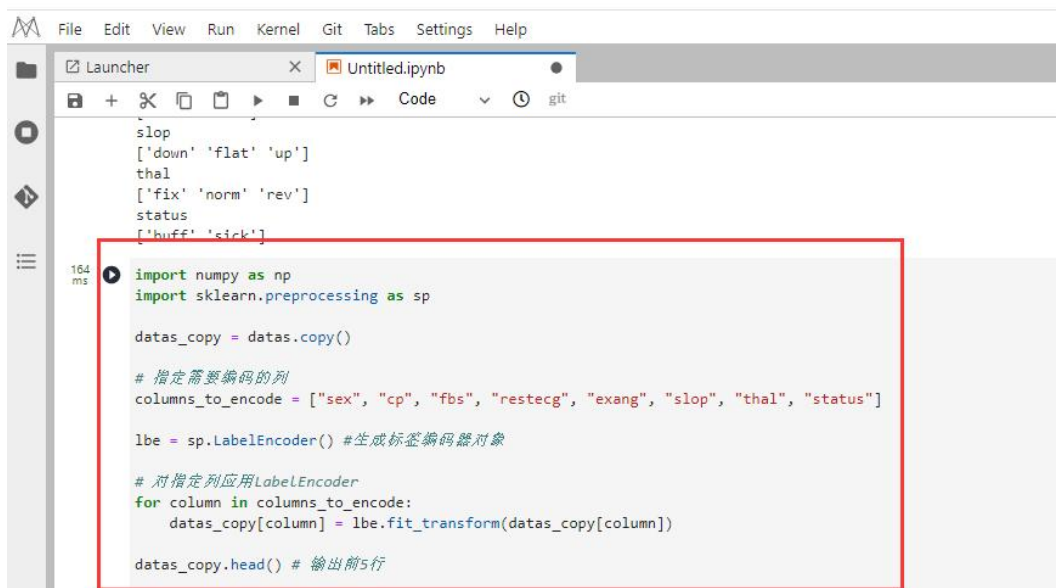
datas_copy = datas.copy()

# 指定需要编码的列
columns_to_encode = ["sex", "cp", "fbs", "restecg", "exang", "slop", "thal", "status"]

lbe = sp.LabelEncoder() #生成标签编码器对象

# 对指定列应用 LabelEncoder
for column in columns_to_encode:
    datas_copy[column] = lbe.fit_transform(datas_copy[column])

datas_copy.head() # 输出前 5 行
```

```

slop
['down' 'flat' 'up']
thal
['fix' 'norm' 'rev']
status
['buff' 'sick']

import numpy as np
import sklearn.preprocessing as sp

datas_copy = datas.copy()

# 指定需要编码的列
columns_to_encode = ["sex", "cp", "fbs", "restecg", "exang", "slop", "thal", "status"]

lbe = sp.LabelEncoder() #生成标签编码器对象

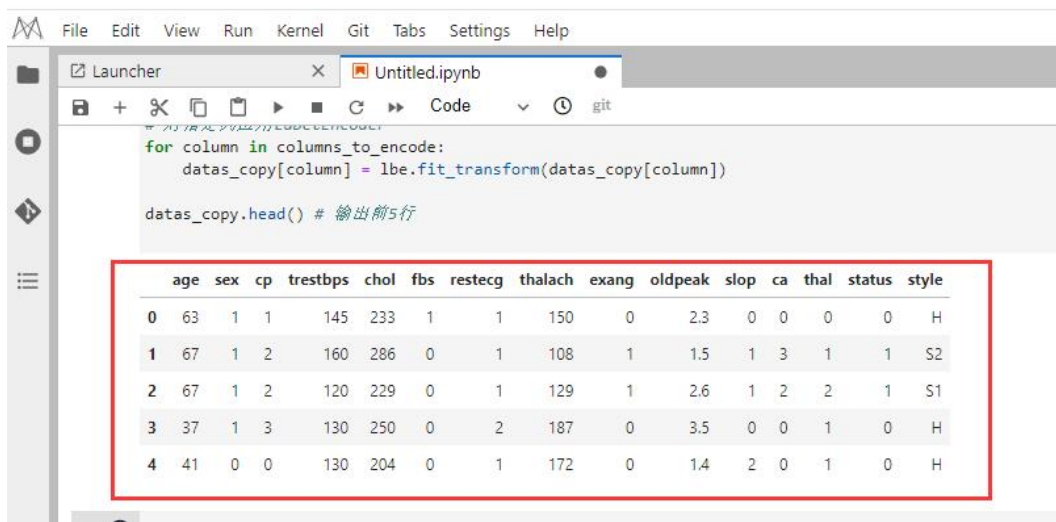
# 对指定列应用LabelEncoder
for column in columns_to_encode:
    datas_copy[column] = lbe.fit_transform(datas_copy[column])

datas_copy.head() # 输出前5行

```

图 3- 1

按 shift+enter 运行单元格，结果如图 3-2 所示。



	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slop	ca	thal	status	style
0	63	1	1	145	233	1	1	150	0	2.3	0	0	0	0	H
1	67	1	2	160	286	0	1	108	1	1.5	1	3	1	1	S2
2	67	1	2	120	229	0	1	129	1	2.6	1	2	2	1	S1
3	37	1	3	130	250	0	2	187	0	3.5	0	0	1	0	H
4	41	0	0	130	204	0	1	172	0	1.4	2	0	1	0	H

图 3- 2

本次运行的结果显示：已将字符类型数据转为了数字类型的数据，例如：sex 字段第 0 行 1 列原为 ‘male’ 转为了 1。

3.2. 归一化

操作步骤如下：

步骤一：对指定字段信息，在单元格中输入以下代码，如图 3-21 所示。


```
# 指定要使用的字段
column_to_nor = datas_copy[
    [
        "age",
        "trestbps",
        "chol",
        "thalach",
        "oldpeak",
        "ca",
        "sex",
        "cp",
        "fbs",
        "restecg",
        "exang",
        "slop",
        "thal",
        "status",
    ]
]
column_to_nor.head() # 输出前 5 行数据
```

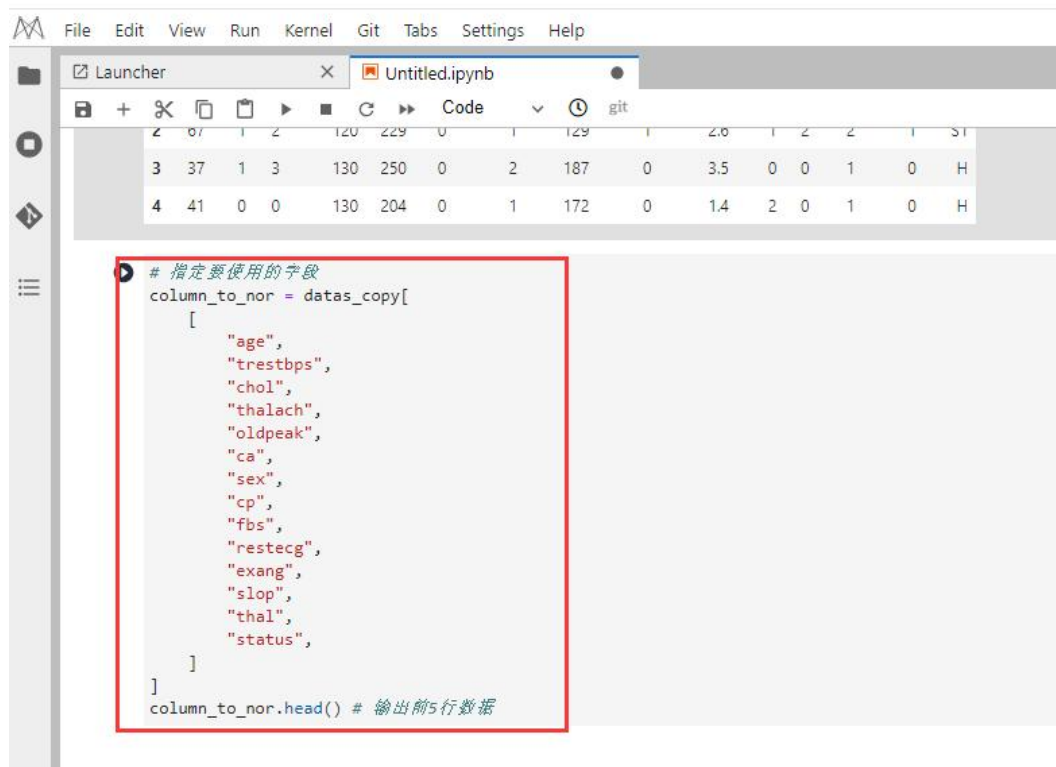
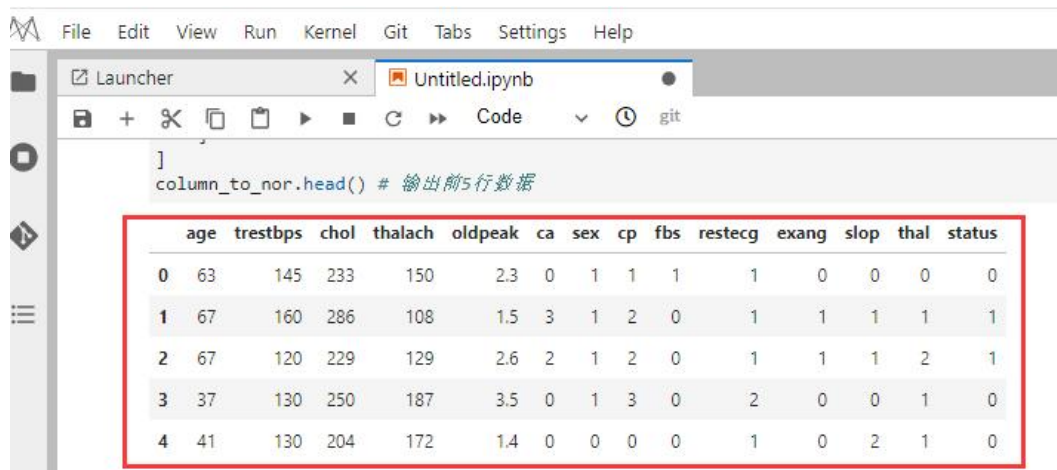


图 3- 3

按 shift+enter 运行单元格，结果如图 3-22 所示。



	age	trestbps	chol	thalach	oldpeak	ca	sex	cp	fbs	restecg	exang	slop	thal	status
0	63	145	233	150	2.3	0	1	1	1	1	0	0	0	0
1	67	160	286	108	1.5	3	1	2	0	1	1	1	1	1
2	67	120	229	129	2.6	2	1	2	0	1	1	1	2	1
3	37	130	250	187	3.5	0	1	3	0	2	0	0	1	0
4	41	130	204	172	1.4	0	0	0	0	1	0	2	1	0

图 3- 4

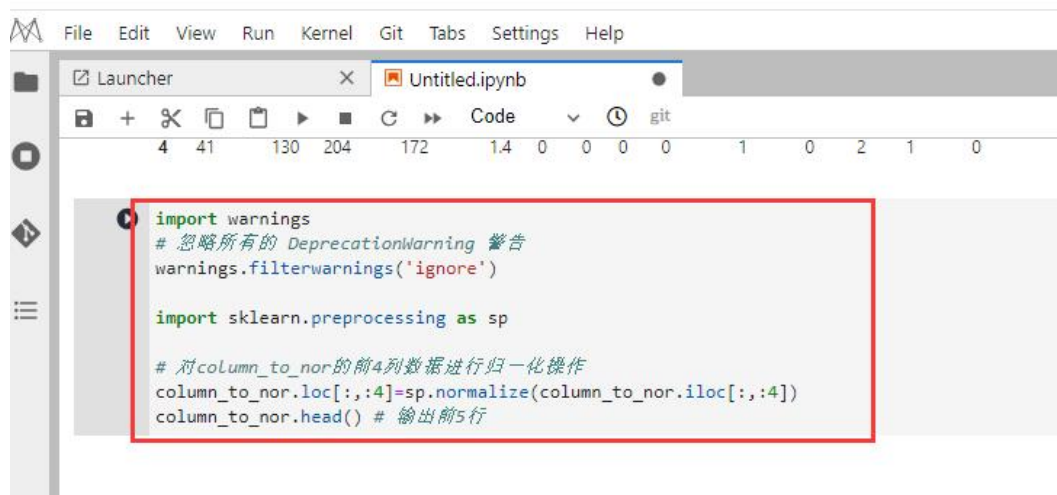
本次运行的结果显示：获取指定字段的前 5 行数据内容。

步骤二：将指定字段进行归一化操作，在单元格中输入以下代码，如图 3-23 所示。

```
import warnings
# 忽略所有的 DeprecationWarning 警告
warnings.filterwarnings('ignore')

import sklearn.preprocessing as sp

# 对 column_to_nor 的前 4 列数据进行归一化操作
column_to_nor.loc[:,4]=sp.normalize(column_to_nor.iloc[:,4])
column_to_nor.head() # 输出前 5 行
```



```
import warnings
# 忽略所有的 DeprecationWarning 警告
warnings.filterwarnings('ignore')

import sklearn.preprocessing as sp

# 对column_to_nor的前4列数据进行归一化操作
column_to_nor.loc[:,4]=sp.normalize(column_to_nor.iloc[:,4])
column_to_nor.head() # 输出前5行
```

图 3- 5

按 shift+enter 运行单元格，结果如图 3-24 所示。

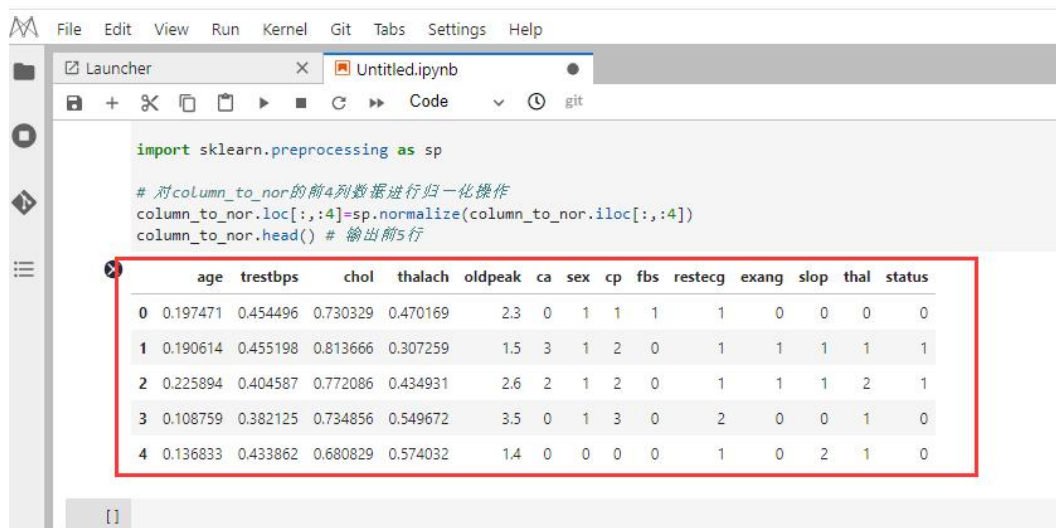


图 3-6

本次运行的结果显示：“age”，“trestbps”，“chol”，“thalach” 四个字段归一化数据。

4. 模型训练和预测

下面将基于体测指标数据，进行建模，并分析模型预测结果。

4.1. 拆分数据集

步骤一：将体测指标数据拆分为训练集和测试集，在单元格中输入以下代码，如图 4-1 所示。

```
# 指定输入集输出集
x = column_to_nor.iloc[:, :-1] # 输入集
y = column_to_nor.iloc[:, -1] # 输出集
print(x.shape)
print(y.shape)
# 拆分训练集和测试集
import sklearn.model_selection as ms

train_x, test_x, train_y, test_y = ms.train_test_split(
    x, y, train_size=0.7, random_state=2
```

```
)
print(train_x.shape)
print(train_y.shape)
```

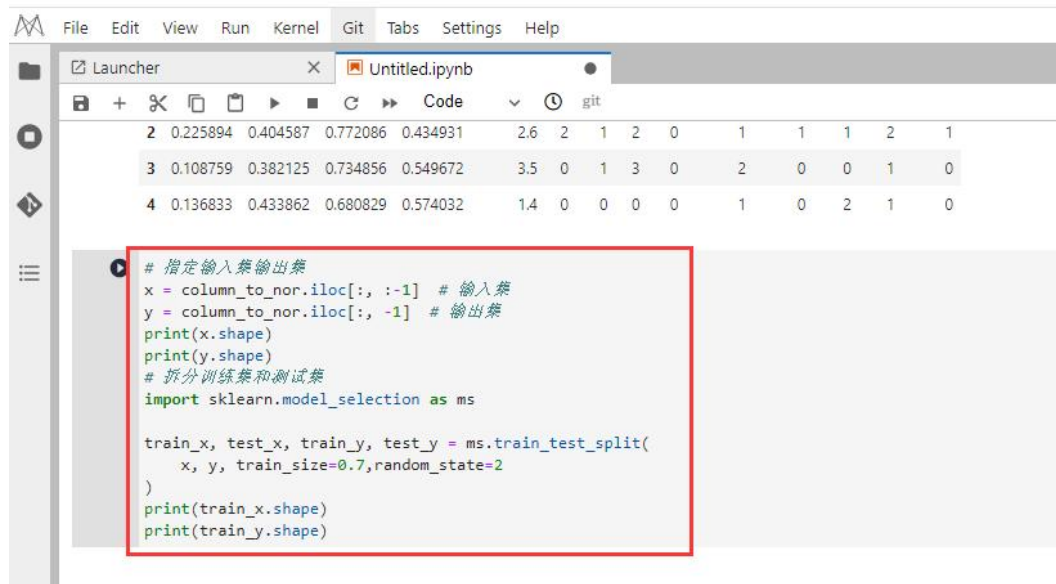


图 4- 1

并按 shift+enter 运行单元格。运行单元格后结果如图 4-2 所示。

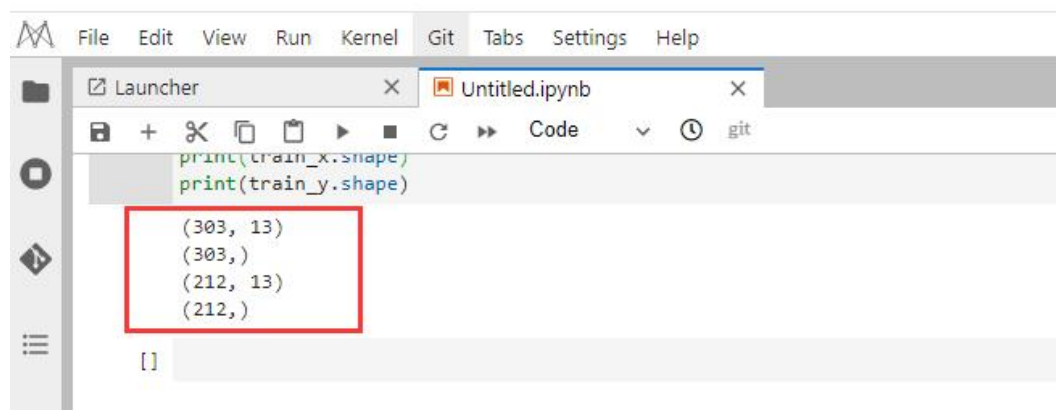


图 4- 2

本次运行的结果显示：将原来的数据拆分为输入集和输出集，并获得各自的 shape。再基于输入集和输出集拆分得到的训练集和测试集数据集 shape。

4.2. 定义模型并预测输出

操作步骤如下：

步骤一：定义逻辑回归分类模型，并基于模型进行预测，在单元格中输入以下代码，如图 4-3 所示。

```
import sklearn.linear_model as lm

# 生成模型对象
model = lm.LogisticRegression(solver="liblinear")

# 使用模型进行训练
model.fit(train_x, train_y)

# 使用模型进行预测
pred_y = model.predict(test_x)

pred_y # 输出预测结果
```

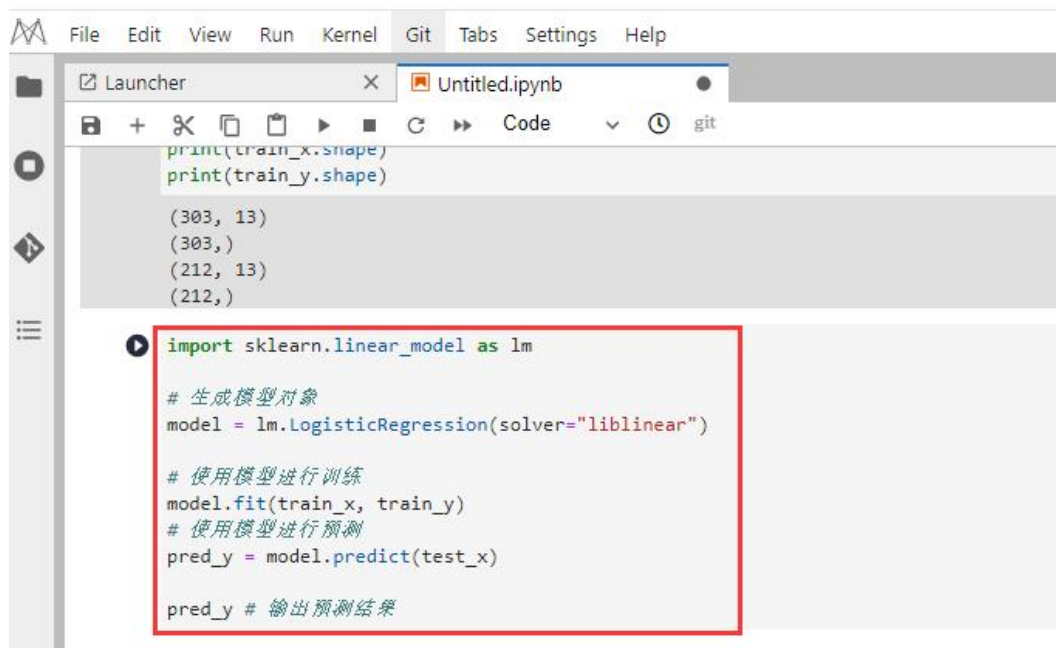
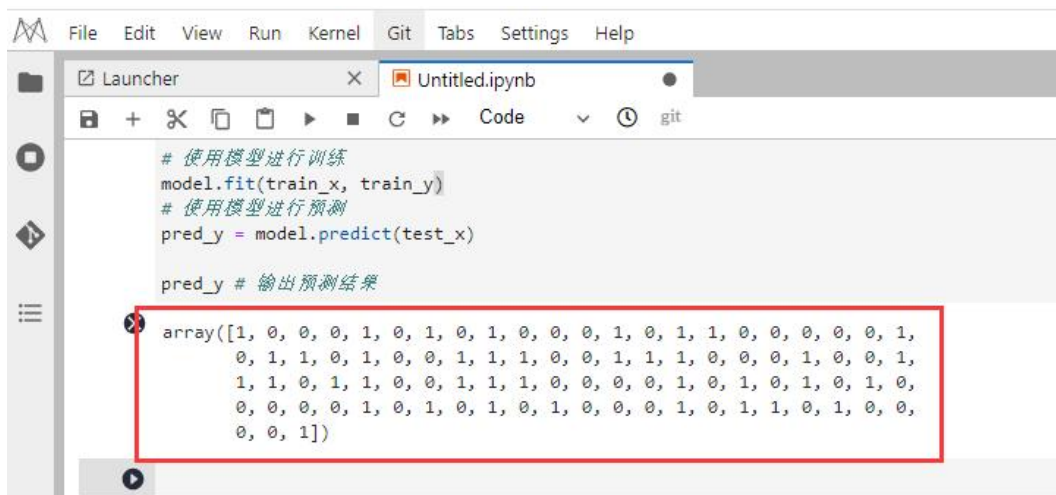


图 4- 3

按 shift+enter 运行单元格，运行单元格后结果如图 4-4 所示。



```

# 使用模型进行训练
model.fit(train_x, train_y)
# 使用模型进行预测
pred_y = model.predict(test_x)

pred_y # 输出预测结果
array([1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1,
       1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0,
       0, 0, 1])
    
```

图 4- 4

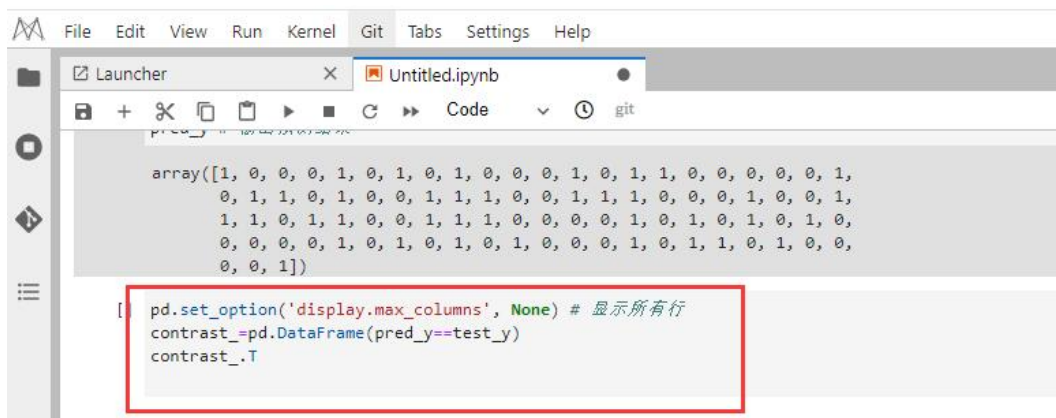
本次运行的结果显示：

- 基于模型进行预测得出的结果。
- 其中 1：表示预测为患病，0：表示预测为健康。

步骤二：查看真实值和预测值对比情况，在单元格中输入以下代码，如图 4-5 所示。

```

pd.set_option('display.max_columns', None) # 显示所有行
contrast_=pd.DataFrame(pred_y==test_y)
contrast_.T
    
```



```

array([1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1,
       0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1,
       1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0,
       0, 0, 1])

[ ] pd.set_option('display.max_columns', None) # 显示所有行
    contrast_=pd.DataFrame(pred_y==test_y)
    contrast_.T
    
```

图 4- 5

按 shift+enter 运行单元格，结果如图 4-6 所示。

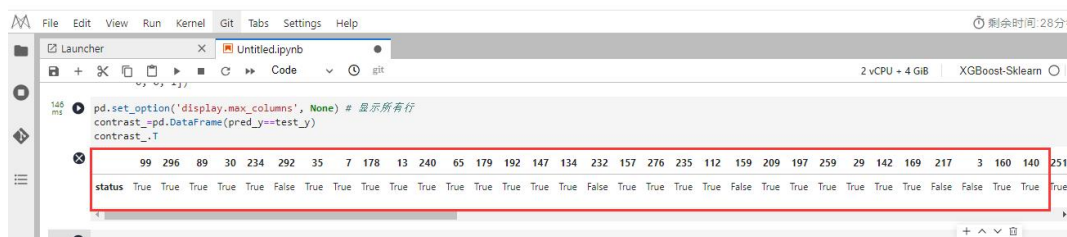


图 4- 6

本次运行的结果显示为：

- True 表示正确的预测结果
- False 表示错误的预测结果

4.3. 查看混淆矩阵

操作步骤如下：

步骤一：获得真实值和预测值的混淆矩阵，在单元格中输入以下代码，如图 4-7 所示。

```
import sklearn.metrics as sm

sm.confusion_matrix(test_y, pred_y) # 输出混淆矩阵
```

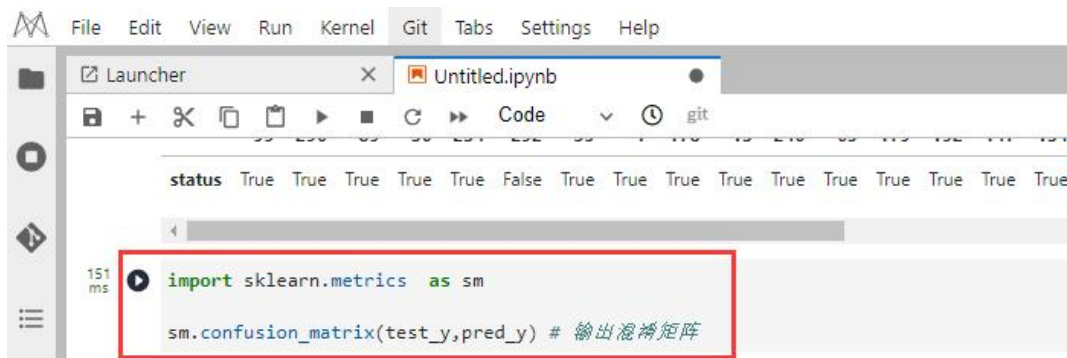
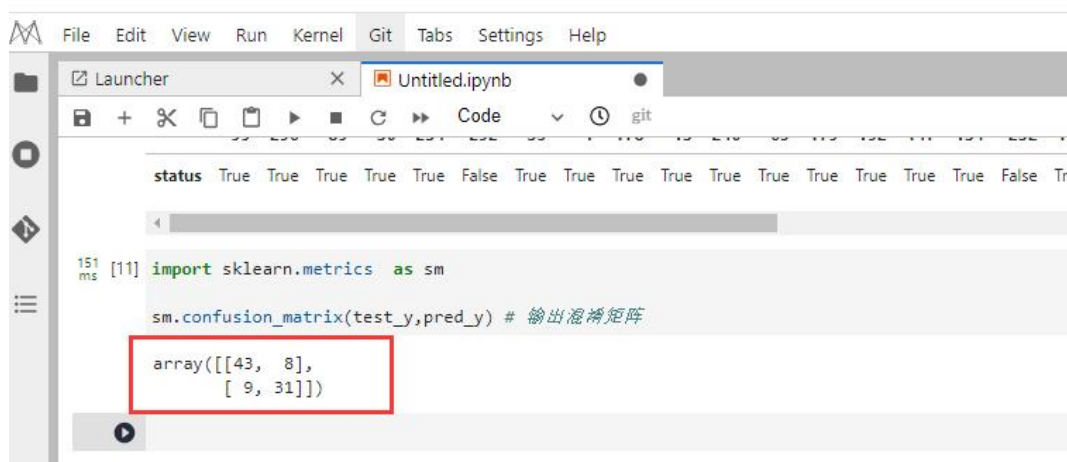


图 4- 7

按 shift+enter 运行单元格，结果如图 4-8 所示。



```

status True True True True True False True True True True True True True True True False Tr
<
151 ms [11] import sklearn.metrics as sm
sm.confusion_matrix(test_y,pred_y) # 输出混淆矩阵
array([[43,  8],
       [ 9, 31]])

```

图 4- 8

本次运行的结果显示为：

- 实际为正类，预测为正类的数量为 43；
- 实际为正类，预测为负类的数量为 8；
- 实际为负类，预测为正类的数量为 9；
- 实际为负类，预测为负类的数量为 31；

4.4. 查看分类报告

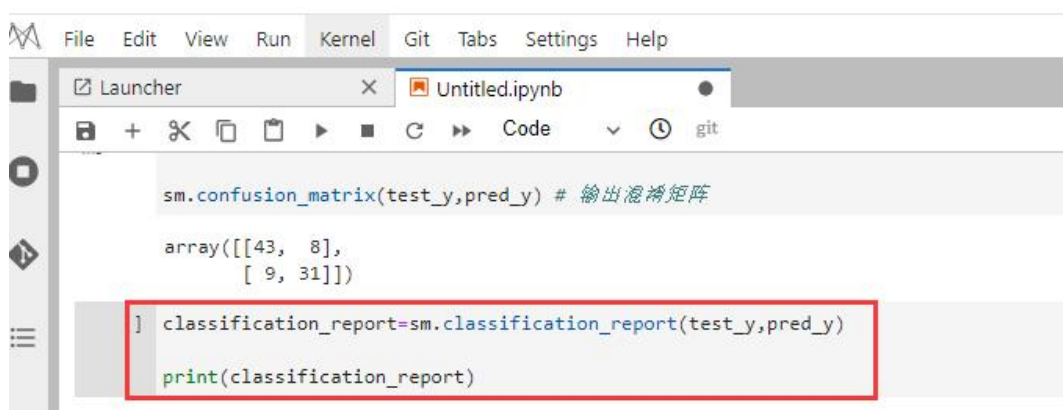
步骤一：获得真实值和预测值的分类报告，在单元格中输入以下代码，如图 4-9 所示。

```

classification_report=sm.classification_report(test_y,pred_y)

print(classification_report)

```



```

sm.confusion_matrix(test_y,pred_y) # 输出混淆矩阵

array([[43,  8],
       [ 9, 31]])

] classification_report=sm.classification_report(test_y,pred_y)
print(classification_report)

```

图 4- 9

按 shift+enter 运行单元格，结果如图 4-10 所示。

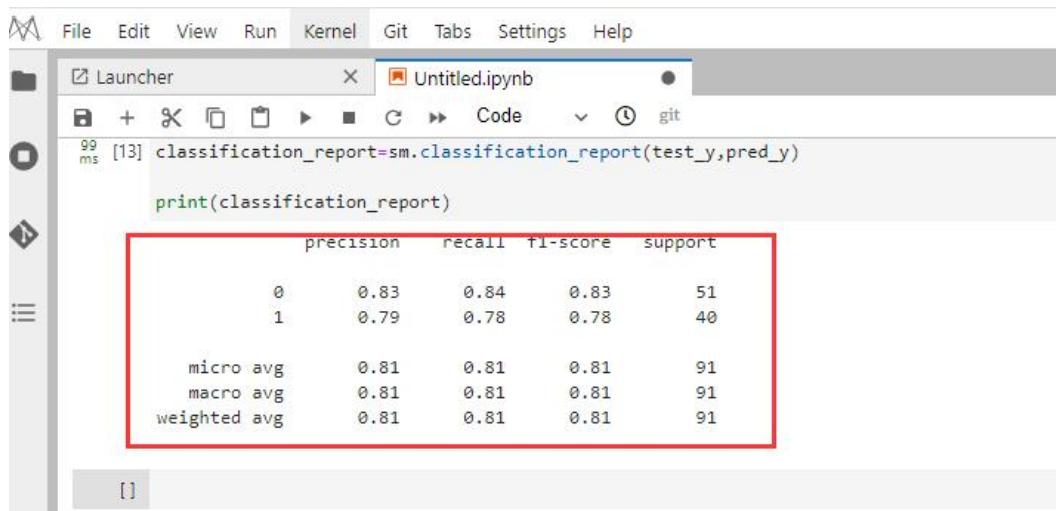


图 4-10

本次运行的结果显示：

- 0 类别的精确度为 0.83，召回率为 0.84，f1-score 为 0.83，support 即数量为 51。
- 1 类别的精确度为 0.79，召回率为 0.78，f1-score 为 0.78，support 即数量为 40。

至此实验结束。

四、实验总结

本实验以体测指标数据为依据，通过 AI 开发平台，并基于逻辑回归分类模型预测分析在指定特征的情况下，患者是否得病的情况。并达到让学员能够熟悉大数据分析挖掘的基本流程，从实验中可以掌握以下知识：

- 数据的上传与读取方式；
- 特征的提取与过滤操作；
- 两种特征工程任务的使用方法；
- 对分类模型的评估方法。